

Frontend Interview Prep Assignment — Set 2 (React + TypeScript)

Overview

Build 5 React + TypeScript features in one GitHub repository. Ship each one as its own pull request, then record one 2-minute video explaining all five. You may use AI tools, but in the interview you must be able to explain the code you submit.

- **Stack:** React and TypeScript (strict mode). Choose your own tooling and libraries: build tool, routing, forms, state, styling and testing.
- **Time limit:** 2 hours for all 5 questions, including PRs and merges. Suggested split: Q1 and Q2 15 minutes each, Q3 and Q4 25 minutes each, Q5 40 minutes. You get 30 more minutes after the cut-off to record and upload the video.
- **What we assess:** working UI, clean and typed components, tests, Git/PR discipline, and above all whether you understand what you built.
- **Rule:** don't use a UI library that does the core work of a question for you.

Claude Code is expected. Use it to write the code fast, and spend the time you save understanding what it wrote. The questions describe *what* to build, not *how*; the design decisions are yours. In the interview, be ready to explain every decision and the alternatives you considered, and to make a short live change.

Submission workflow

Each question is one branch, one PR and one merge, in order from Q1 to Q5.

1. Create a public GitHub repo named `fe-interview-prep`. On `main`, set up a React + TypeScript app with a linter and a test runner. Each question lives on its own page or route.
2. For each question, create a branch from the latest `main`: `feature/q1-cart`, `feature/q2-feed`, `feature/q3-kanban`, `feature/q4-dashboard`, `feature/q5-comments`.
3. Commit in small, meaningful steps, such as `Add cart quantity controls` or `Stop duplicate page loads`. Avoid commits like `fix`, `changes` or `final`.
4. Open a PR into `main` using the template below. Add screenshots or a GIF of the UI. Review your own diff before merging.
5. Merge the PR, then pull `main` before starting the next branch.
6. After the last merge, record the video, upload it to YouTube and add the link to the README.

PR description template

Problem

What this PR solves (1–2 lines).

Approach

How the components and state are organised.

Decisions & trade-offs

What you chose, and why over the alternatives.

Screenshots

UI before/after or a GIF.

How to test

Page to open, steps, test files.

README (keep it updated as you merge; add the video link at the end):

#	Question	PR link
1	Shopping Cart	
2	Infinite Feed	
3	Kanban Board	
4	Live Dashboard	
5	Comments with Offline Support	

Video:

Video guidelines

After the last merge, record **one screen recording of at most 2 minutes** covering all 5 questions. Upload it to YouTube as **Unlisted** and add the link to the README.

Time	What to show
0:00–0:10	Quick tour of the repo and the merged PRs
0:10–1:50	About 20 seconds per question: what it does, and the most important decision you made
1:50–2:00	One thing you would improve with more time

- With 20 seconds per question, skip the line-by-line walkthrough. Show the one piece of code that matters most and say *why* you built it that way.
- Use any recorder: Loom, OBS, QuickTime or the built-in screen recorder.
- Keep your face camera optional and your voice clear. Zoom the editor font to at least 16px.

AI usage policy

AI tools (Claude, Copilot, ChatGPT) are allowed. Not understanding your own code is not. The interviewer will open your PRs and ask follow-up questions like these:

- Why does this component re-render? Could you prevent it?
- Why is this value stored rather than calculated?
- What happens if the network is slow, or the user leaves the page mid-request?
- Why did you choose this library or approach? What alternatives did you consider?
- Change this behaviour live.

If you can't answer these about your own code, the question counts as not done.

The questions

Q1 — Shopping Cart

Build a product list with a shopping cart.

Requirements

- Show products from `https://dummyjson.com/products` with name, price, image and stock.
- Add products to the cart, change quantities and remove items. A quantity can't exceed the product's stock.
- Show the subtotal, an 18% tax and the total. Amounts always display correctly to 2 decimal places.
- The cart survives a page refresh.
- Show an empty-cart state.

Acceptance criteria

- Changing a quantity updates every total immediately.
- At least one automated test.

Q2 — Infinite Feed

Build a feed of posts that loads more as the user scrolls.

Requirements

- Load posts from `https://dummyjson.com/posts`, 10 at a time.
- Load the next page automatically when the user nears the bottom.
- Scrolling fast must never load the same page twice or show duplicate posts.
- Show loading, error (with retry) and "You've reached the end" states.
- Clicking a post opens its detail page. Going back returns the user to the same scroll position.

Acceptance criteria

- Scrolling quickly to the bottom several times produces no duplicate requests or posts.
- At least one automated test.

Q3 — Kanban Board

Build a task board with three columns: To do, In progress and Done.

Requirements

- Add, edit and delete cards. A card has a title (required) and an optional description.
- Move cards between columns and reorder them within a column by dragging.
- Users who can't use a mouse can still move cards (for example, with the keyboard or a menu).
- Show the number of cards in each column.
- The board survives a page refresh.

Acceptance criteria

- Moving a card updates both columns and their counts correctly.
- At least one automated test.

Q4 — Live Dashboard

Build a dashboard whose widgets refresh with live data.

Requirements

- Mock an API (any approach) that returns sales, active users and recent orders, with values that change on every call.
- Show at least 3 widgets: a sales number, an active-users chart and a recent-orders list.
- Refresh the data every 5 seconds. Pause refreshing while the browser tab is hidden, and resume when it's visible again.
- Newer data on screen is never replaced by an older, late response, and requests don't pile up when the API is slow.
- Users can show or hide widgets, and their choice survives a refresh.
- Updating one widget's data should not re-render the widgets whose data didn't change.

Acceptance criteria

- Switching to another tab stops the requests (visible in the Network tab), and they resume on return.
- At least one automated test.

Q5 — Comments with Offline Support

Build a comment thread that feels instant and keeps working without a network connection.

Requirements

- Mock a comments API (any approach) that is slow (1–2 seconds) and fails about 20% of the time.
- A new comment appears immediately when posted, marked as "sending" until the server confirms it.
- If sending fails, the comment stays visible, marked as failed, with a retry button.
- While offline, new comments are queued and sent automatically, in order, when the connection returns.
- Queued comments survive a page refresh.

- Retries must never create duplicate comments on the server.

Acceptance criteria

- Going offline, posting 3 comments and coming back online sends all 3 in order, with no duplicates.
- At least one automated test.

If you finish early (optional)

None of these are required. Finish all 5 core questions first.

- Q1: a discount code field.
- Q2: a "back to top" button and pull-to-refresh.
- Q3: a limit of 3 cards in In progress.
- Q4: let users rearrange widgets, and remember the layout.
- Q5: let users edit a comment while it's still queued.

Submission checklist

- ☐ Public repo `fe-interview-prep` created, with a README
- ☐ 5 branches, 5 PRs, all merged into `main`
- ☐ Every PR uses the description template and includes screenshots or a GIF
- ☐ The README explains how to run the app and the tests
- ☐ Tests, lint and type-check all pass
- ☐ Code and PRs done within 2 hours; one video of at most 2 minutes uploaded to YouTube (Unlisted) within 30 minutes after
- ☐ The README has every PR link and the YouTube link
- ☐ You can explain every file in the repo without AI help